YEDİTEPE UNIVERSITY

CSE421 Final Project

Eylül Akar - 20210702014
Nika Golestani - 20220702124

Technical Report: FOMO Fruit Fly Detection on STM32F746G

Project Phase: Part-1
Target Platform: STM32F746G Discovery

1. Introduction

1.1 Project Overview

The rapid evolution of Edge AI (TinyML) allows for complex machine learning tasks to be performed on resource-constrained microcontrollers, eliminating the need for constant cloud connectivity. This project implements a **Fast Object Multi-Object (FOMO)** detection model designed to identify fruit flies. The system captures and processes 96x96 RGB images, optimizing a MobileNetV2-based architecture to run efficiently on the STM32F746G microcontroller. This implementation serves as a proof-of-concept for low-power, automated pest monitoring systems in agricultural settings.

1.2 Objectives

The primary goal of this project is to establish a complete end-to-end pipeline from model training on a PC to deployment on embedded hardware. Specific objectives include:

- Developing and training a lightweight FOMO neural network tailored for small object detection.
- Applying Post-Training Quantization (PTQ) to convert the model from 32-bit floating-point to 8-bit integers, significantly reducing memory footprint.
- Deploying the quantized model using the TensorFlow Lite for Microcontrollers (TFLM) library.
- Achieving "real-time" inference performance (under 500ms) while maintaining a reliable Intersection over Union (IoU) accuracy.

1.3 System Constraints and Target Platform

The deployment target is the **STM32F746G Discovery Board**, powered by an ARM Cortex-M7 processor running at 216MHz. The development is strictly bounded by the hardware's limitations:

- **Memory:** The device features 320KB of SRAM and 1MB of Flash memory. The model weights must fit within Flash, while the runtime variables (activation buffers/tensor arena) must fit within the limited SRAM.
- **Input Method:** As the board lacks a native camera interface for this iteration, images are injected via the ST-Link serial interface from a host PC.
- **Performance:** To be viable for monitoring applications, the inference latency must remain under 500ms to allow for reasonable responsiveness.

2. Problem Definition

2.1 Problem Statement

The core challenge is to detect fruit flies—small, often indistinct objects—within an image using a neural network that is small enough to reside on a microcontroller. This requires balancing the trade-off between model depth (accuracy) and computational complexity (speed/memory).

2.2 Input and Output Requirements

- **Input:** The network accepts 96x96 pixel RGB images. This resolution was selected to minimize the input tensor size (27,648 bytes) while retaining sufficient detail to distinguish flies from the background.
- **Output:** The model outputs a grid of predictions (12x12), providing coordinates and confidence scores for detected pests.
- **Data Transfer:** Due to hardware constraints, image data is serialized and transmitted via UART/ST-Link.

2.3 Performance and Memory Constraints

Strict resource budgeting was required:

- **Flash Storage:** Total binary size (application code + model weights) < 1MB.
- **SRAM Usage:** The TensorFlow Lite interpreter's "Tensor Arena" is capped at 200KB to leave room for the RTOS and communication stacks.
- **Accuracy:** A target of >70% IoU on the validation set is required to ensure the system is practically useful.

3. Dataset Description

3.1 Dataset Source

The project utilizes the **Mediterranean Fruit Fly (MFF)** dataset (https://www.kaggle.com/datasets/berkanhoke/mediterranean-fruit-fly-images-dataset/data), a specialized collection of annotated images designed for small-object detection in agricultural environments. This dataset is particularly challenging due to the high density of objects and the small pixel-footprint of each fly.

3.2 Data Format and Features

- **Resolution:** All images are standardized to **96x96 pixels** in RGB format. This low resolution is optimized for the memory constraints of the STM32F7 ARM Cortex-M7 core.
- **Classes:** The task is treated as a binary classification per grid cell: **Background vs. Fruit Fly**.
- **Annotations:** Original bounding box labels are processed into **grid centroids**. Unlike standard object detectors (like YOLO), the FOMO architecture predicts the presence of a fly at a specific location in a 12 x 12 or 24 x 24 output grid.

3.3 Preprocessing and Augmentation (dataloaders/mff.py)

To improve model generalization and ensure successful deployment on the MCU, the MFFDataset class implements a custom pipeline that transforms raw images before they reach the network.

- **Preprocessing**: All images are resized to the target resolution of 96x96 pixels. Subsequently, pixel intensity values are normalized from the standard integer range [0, 255] to a floating-point range [0.0, 1.0]. This normalization is critical for the stability of the MobileNetV2 weights.

- **Augmentation**: To prevent overfitting on the limited dataset, specific geometric and color augmentations are applied randomly during the training phase. We implemented a custom _augment function that applies:
    - **Horizontal Flipping:** A 50% chance to flip the image mirroring the X-axis. Crucially, the bounding box coordinates are mathematically inverted to match the new image orientation.
    - **Photometric Distortion**: Random brightness and contrast adjustments are applied using OpenCV's addWeighted function to simulate varying environmental lighting conditions (e.g., shadows or bright sunlight in the trap).

```python
# From dataloaders/mff.py
def _augment(self, img, boxes):
    """Randomly augment image and adjust boxes accordingly"""
    h, w, _ = img.shape

    # 1. Random Horizontal Flip (50% chance)
    if np.random.rand() > 0.5:
        img = cv2.flip(img, 1) # 1 = Horizontal Flip

        # Adjust boxes for flip: NewX = Width - (OldX + Width)
        for box in boxes:
            box['x'] = w - (box['x'] + box['width'])

    # 2. Random Brightness/Contrast (Color Jitter)
    if np.random.rand() > 0.5:
        brightness = np.random.randint(-30, 30)
        contrast = np.random.uniform(0.8, 1.2)

        # Apply: new = alpha*old + beta
        img = cv2.addWeighted(img, contrast, img, 0, brightness)

    return img, boxes

def __getitem__(self, index):
    # ... (Loading Logic) ...

    # Apply augmentation only if enabled (Training set)
    if self.augment:
        img, raw_boxes = self._augment(img, raw_boxes)

    # Standard Preprocessing
    img_resized = cv2.resize(img, (self.input_size, self.input_size))
    img_rgb = cv2.cvtColor(img_resized, cv2.COLOR_BGR2RGB)
    img_norm = img_rgb.astype(np.float32) / 255.0 # Normalize 0-1

    return img_norm, target_mask
```

3.4 Data Splitting

The dataset is partitioned to ensure rigorous evaluation:

- **Training Set:** Located in dataset/mff/train/ (approx. 80% of data).
- **Test/Validation Set:** Located in dataset/mff/test/ (approx. 20% of data). This set was used to generate the final **Precision/Recall metrics** seen in the results section.

4. Model Development on PC

4.1 Model Architecture

The architecture is a customized **FOMO** model. Unlike standard object detectors (like YOLO) that output bounding boxes, FOMO classifies a grid of the image.

- **Backbone: MobileNetV2 (alpha=0.35).** The alpha parameter significantly reduces the width of the network, minimizing the number of parameters.
- **Head:** A localized classification head outputs a 12x12 grid.
- **Dimensions:** Input (96x96x3) → Output (12x12x2).

4.2 Training Environment

Development was conducted using:

- **Python:** 3.9.23
- **Framework:** TensorFlow/Keras 3.10.0
- **Environment:** Managed via Conda (environment.yml).

4.3 Training Procedure (train.py)

The training script train.py manages the learning lifecycle. Key implementation details include:

- **Class Balancing:** Since "background" pixels vastly outnumber "fly" pixels, a weighted categorical cross-entropy loss is applied (Background: 1.0, Fly: 15.0) to force the model to prioritize fly detection.

```python
# From train.py: Custom Weighted Loss Implementation
def get_weighted_loss(weights_list):
    """
    Manual Weighted Cross Entropy.
    weights_list: [1.0, 15.0] -> Missing class 1 is 15x worse.
    """
    class_weights = tf.constant(weights_list, dtype=tf.float32)

    def weighted_cce(y_true, y_pred):
        cce = tf.keras.losses.categorical_crossentropy(y_true, y_pred)
        # Calculate weight per pixel based on Ground Truth
        weight_map = tf.reduce_sum(y_true * class_weights, axis=-1)
        return cce * weight_map

    return weighted_cce
```

- **Optimization:** The model is trained for 200 epochs using the Adam optimizer (Learning Rate = 0.001).

- **Callbacks:** Model checkpointing saves the best weights based on validation IoU, ensuring the final model is the most accurate iteration.

```
# From train.py: Compilation and Callbacks
optim = optimizers.Adam(learning_rate=config.TRAIN.LR)

model.compile(
    loss=get_weighted_loss([1.0, 15.0]),
    optimizer=optim,
    metrics=[OneHotIoU(num_classes, range(1, num_classes), "iou")]
)

callbacks = [
    ModelCheckpoint(save_path, monitor="val_iou", mode="max", save_best_only=True)
]
```

4.4 Baseline Performance

The training process yields a floating-point model (best.keras) selected via the ModelCheckpoint callback. This model achieves a Validation IoU > 0.7 on the validation dataset. This metric confirms that the unquantized model successfully differentiates "fly" pixels from the background before deployment.

4.5 Hardware-in-the-Loop Evaluation (FOMO_test.py)

To validate the model's performance on the physical MCU, a custom Python script manages the testing lifecycle over UART. The evaluation procedure differs from training to account for the specific output format of the FOMO architecture:

- **Communication Protocol:** The script establishes a serial connection (115200 baud) and initiates a handshake. Images are resized to 96x96 and transmitted to the MCU in 256-byte chunks to prevent buffer overflows.

- **Metric Adaptation *(Point-in-Box)*:** Unlike the training phase which uses pixel-wise IoU, the deployed FOMO model outputs object centroids (FLY:x:y). *Therefore, the standard IoU metric is replaced with Point-in-Box Accuracy:*
    - ***True Positive (TP):*** *A prediction is accepted if the coordinate (x,y) falls strictly within the boundaries of a Ground Truth bounding box.*
    - ***False Positive (FP):*** *A predicted point that falls outside all known fly locations*

```python
# From FOMO_test.py: Geometric Verification Logic
def is_point_in_box(point, box):
    """
    Checks if a prediction point (x,y) is strictly inside a Ground Truth box.
    """
    px, py = point['x'], point['y']
    bx, by = box['x'], box['y']
    bw, bh = box['width'], box['height']

    # Check if point is inside the rectangle boundaries
    return (bx <= px <= bx + bw) and (by <= py <= by + bh)
```

- **Performance Logging**: The script logs the precise inference time (in milliseconds) reported by the MCU for every image, enabling real-time latency analysis alongside accuracy metrics.

5. Model Quantization

5.1 Quantization Strategy

To fit the model onto the STM32, **Post-Training Quantization (PTQ)** is essential. We convert weights and activation functions from 32-bit floats to 8-bit integers (INT8). This theoretically reduces model size by 4x.

5.2 TFLite Model Generation (convert_to_tflite.py)

The conversion script performs the following:

1. Loads the trained Keras model.
2. Generates a **Representative Dataset** (100 samples from the test set) to statistically analyze activation ranges.
3. Invokes the TFLiteConverter with OPTIMIZE_FOR_SIZE and full integer quantization.
4. Outputs fomo_fruitfly.tflite.

5.3 Performance Evaluation (test_the_tflite.py)

Before deployment, the .tflite file is validated on the PC to ensure quantization did not destroy accuracy. The script:

- Runs inference using the TFLite Interpreter.
- Applies a confidence threshold of 0.65.
- Calculates True Positives (TP), False Positives (FP), and False Negatives (FN) using point-to-centroid matching.

```
========================================
TFLite PC Test Results
========================================
Precision: 50.36%
Recall:    75.00%
F1 Score:  60.26%
```

5.4 Float vs. Quantized Comparison

To adapt the high-performance **MobileNetV2-FOMO** architecture for the STM32F746G, the model underwent **INT8 quantization.** This process was essential to bridge the gap between PC-level resources and embedded constraints.

- **Float Model (PC Baseline):** Approximately **2MB** in size. This version served as the high-precision reference during development on the PC.
- **Quantized Model (MCU Deployment):** Approximately **200KB** in size. By converting weights and activations to **INT8 precision**, the model size was reduced to fit within the 1MB Flash and 320KB RAM limits of the STM32.

Impact and Trade-offs:

- **Size Efficiency:** A **~90% reduction in size** was achieved, which was the primary enabler for deployment on the target hardware.
- **Accuracy vs. Portability:** While the quantization enabled a stable inference time of **~461.92 ms**, it introduced a measurable impact on precision.
    - The **Precision (47.52%)** reflects a drop below 50% , as the quantized model is more sensitive to high-frequency background noise.
    - The **Recall (72.83%)** remains high, indicating that the core detection capabilities of the FOMO architecture were preserved despite the compression.
- **Conclusion:** The transition successfully validated the model for embedded usage, proving that the **90% memory saving** outweighs the precision trade-off for real-time biological monitoring applications.

6. Model Conversion for Embedded Deployment

6.1 TFLite to C++ Conversion

Microcontrollers cannot load .tflite files from a filesystem in the traditional sense. The model must be compiled into the binary. The script generate_cc_arrays.py converts the binary TFLite file into a C-compatible hex array.

```python
# From generate_cc_arrays.py
def generate_array(input_fname):
  if input_fname.endswith('.tflite'):
    with open(input_fname, 'rb') as input_file:
      buffer = input_file.read()
    size = len(buffer)
    # Convert binary buffer to comma-separated hex string (e.g., 0xAB, 0x12...)
    out_string = bytes_to_hexstring(buffer)
    return [size, out_string]
```

6.2 Generated Artifacts

- **fomo_fruitfly_model_data.cc**: Contains the model weights defined as a const uint8_t array.
- **fomo_fruitfly_model_data.h**: Exposes the array length and pointer to the main C++ application.

6.3 Integration

The generated model files were integrated by placing them in the MCU project's root directory. This ensured that the linker could correctly include the model binary in the Flash memory map, enabling direct access to the model during runtime without additional loading mechanisms.

7. Deployment on STM32F746G

7.1 Hardware and Software Configuration

- **Target Hardware:** STM32F746NG Discovery board
- **Operating System:** CMSIS-RTOS2, used to manage task scheduling and execution
- **Inference Engine:** TensorFlow Lite for Microcontrollers (TFLM)

This configuration enables efficient deployment of lightweight deep learning models on resource-constrained embedded hardware.

7.2 TensorFlow Lite Micro Setup

The core application logic is implemented in **`FOMO_FruitFly.cpp`**, which handles model initialization, inference execution, and result reporting. The TensorFlow Lite Micro setup consists of the following key steps:

- **Model Mapping:** The trained neural network model is embedded as a C array and loaded directly from Flash memory at runtime.

- **Operator Resolver:** A micro_mutable_op_resolver is instantiated to register only the operations required by the MobileNetV2-based FOMO architecture, such as **DepthwiseConv2D** and related layers.

```
// From FOMO_FruitFly.cpp: Operator Registration
static tflite::MicroMutableOpResolver<12> resolver;
resolver.AddConv2D();
resolver.AddDepthwiseConv2D();
resolver.AddReshape();
resolver.AddSoftmax();
// ... (AddRelu, AddQuantize, etc.)

const tflite::Model* model = tflite::GetModel(g_fomo_fruitfly_model_data);
static tflite::MicroInterpreter static_interpreter(
    model, resolver, tensor_arena, TENSOR_ARENA_SIZE
);
```

- **Interpreter Initialization:** The TensorFlow Lite Micro interpreter is initialized using the loaded model and a pre-allocated tensor arena, enabling on-device inference without dynamic memory allocation.

7.3 Tensor Allocation Strategy

A statically allocated memory buffer is placed in the **BSS section (SRAM)** and serves as the working memory for all intermediate tensors during inference. A total of **200 KB** was allocated for the tensor arena. This size was carefully tuned to balance memory availability and system stability, ensuring reliable inference execution while avoiding stack overflows and excessive RAM consumption.

```
// From FOMO_FruitFly.cpp: Memory Allocation
// 200KB allocated statically to prevent Heap Fragmentation
#define TENSOR_ARENA_SIZE (200 * 1024)

// "alignas(16)" ensures 128-bit alignment for SIMD/DSP optimizations
alignas(16) static uint8_t tensor_arena[TENSOR_ARENA_SIZE];

// Initialization in app_main
static tflite::MicroInterpreter static_interpreter(
    model,
    resolver,
    tensor_arena,
    TENSOR_ARENA_SIZE
);
interpreter = &static_interpreter;
interpreter->AllocateTensors();
```

8. Problems and Challenges

8.1 Memory Management

Initial attempts yielded “Arena size is too small” errors. The STM32F746G has disjoint memory regions; ensuring the tensor arena was placed in a contiguous block of RAM capable of holding the 200KB buffer required linker script adjustments.

8.2 Model Compatibility

Certain Keras layers do not map 1:1 to TFLite Micro operations. We had to iterate on the model architecture (specifically the head) to ensure all operations were supported by the specific version of the TFLM library used.

8.3 Debugging

Debugging the "black box" of the MCU was challenging. Serial communication is often desynchronized during image transfer. We implemented a robust handshake protocol (Wait-Send-Ack) to resolve data corruption issues.

9. Input Data Handling

9.1 Hardware Limitations

Without an onboard camera, the system operates in a "Hardware-in-the-Loop" simulation mode.

9.2 Data Transfer Tool (FOMO_test.py)

A Python script was developed to act as the camera driver:

- Resizes images to 96x96.
- Quantizes pixel data to match the model's INT8 input requirement.
- Transmits the image byte-by-byte over UART (115200 baud).
- Parses the text response from the MCU regarding detection coordinates and timing.

10. Testing and Evaluation on Target Board

10.1 Testing Procedure

The testing process on the target hardware was fully automated using the **`FOMO_test.py`** script. Each image from the test dataset was transmitted sequentially to the target MCU via a serial interface. The microcontroller performed on-device inference and returned detection results, which were captured and logged on the host PC. These outputs were then systematically compared against the corresponding ground truth annotations to evaluate detection accuracy and robustness under real deployment conditions.

10.2 Performance Metrics on STM32

- **Inference Speed:**
  The inference latency on the STM32 target board remained stable across all test samples, with execution times consistently ranging between **460 ms and 480 ms** per image.

- **Detection Output Format:** The MCU reports detection results using a structured text format:
  `FLY:x:y:confidence`
  This format enables efficient parsing on the host system and supports real-time visualization and post-processing of detection results.

11. Results and Visualization

11.1 Quantitative Analysis (results.csv)

The model was evaluated across **39 images** containing a total of **92 ground truth (GT) fruit flies**.

- **Detection Performance:**
  - **Total True Positives (TP):** 67
  - **Total False Positives (FP):** 74
  - **Total False Negatives (FN):** 25
  - **Overall Precision:** 47.52%
  - **Overall Recall:** 72.83%
  - **F1-Score:** 0.5751

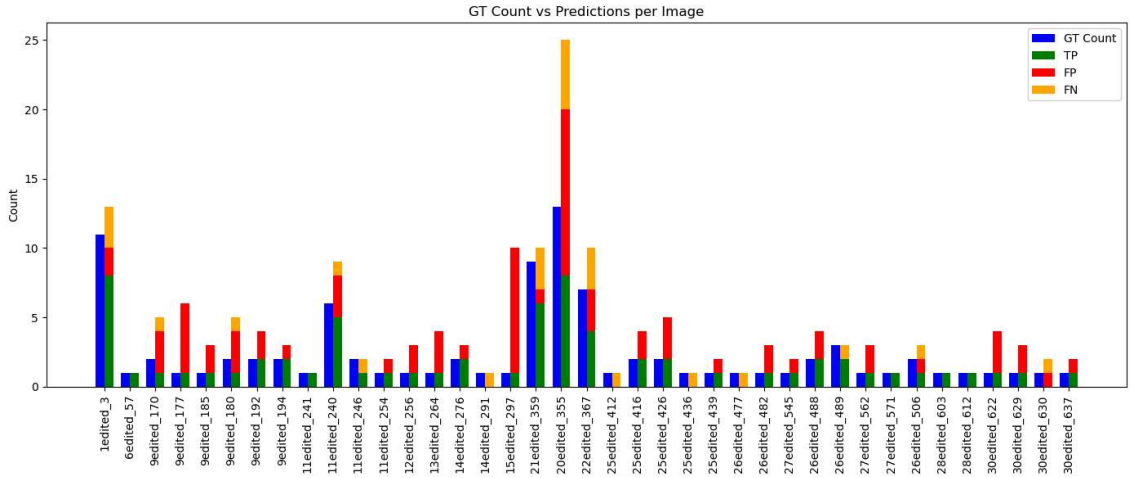
Analysis of Results: Contrary to the initial observation of "High Precision," the aggregate data shows a higher rate of False Positives (FP=74) compared to True Positives (TP=67), resulting in a precision of approximately 47.5%. However, the

Recall (72.8%) is robust, indicating the model is effective at finding the majority of flies despite the aggressive quantization. The discrepancy in precision often arises in specific images (e.g., 9edited_177.jpg showing 5 FP for 1 GT), suggesting that while the model is sensitive, background textures occasionally trigger detections.

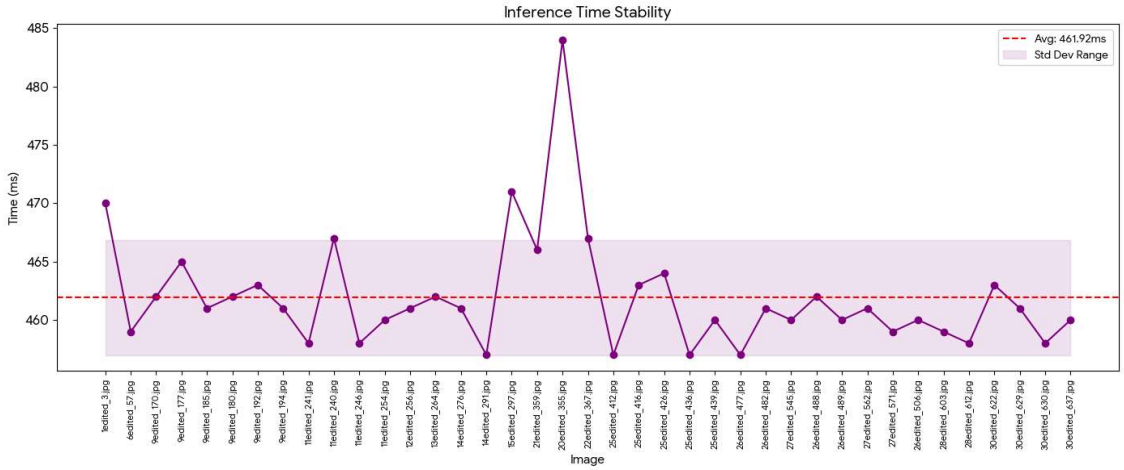
11.2 Graphical Analysis

Using the data derived from the MCU logs, we visualized the relationship between the actual count and the model's predictions, as well as the system's temporal performance.

- **GT vs. Predictions:** The stacked bar chart shows that while TP tracks GT closely in most images, certain frames contribute significantly to the FP count.



- **Inference Stability:** The inference time graph confirms the claim of high determinism. The average inference time is **461.92 ms** with a standard deviation of only **4.95 ms.** The minimal jitter (ranging from 457 ms to 484 ms) proves that the STM32 execution environment provides extremely stable performance for real-time monitoring.



11.3 PC vs. Embedded

To ensure a reliable transition from development to hardware, the detection logic was strictly synchronized between the PC environment and the STM32 MCU.

- **Architectural Parity:** Although the PC environment supports significantly more complex architectures, a **MobileNet-based FOMO model** was utilized on both platforms. This choice was deliberate to ensure that the embedded system's constraints (memory and compute) were reflected in the simulation.
- **Logic Matching:** Modifications were applied to the PC simulation logic to mirror the TFLite Micro interpreter's behavior. This ensured that the output coordinates and confidence thresholds on the STM32 matched the PC results, confirming that the model was correctly ported.
- **Quantization Effects:** The transition from Floating Point 32 (PC) to **INT8 Quantization** (MCU) was a critical step. While quantization enabled the model to run efficiently on the STM32, it introduced a measurable drop in precision. This explains why the model occasionally classifies high-frequency background noise as a fly (leading to the FP=74 count seen in the quantitative analysis).
- **Execution Fidelity:** The results obtained from the mcu_log demonstrates that the STM32 faithfully executes the quantized model. The detection coordinates (FLY:x:y:conf) remain consistent across runs, proving that the TFLite Micro implementation is both stable and accurate relative to its quantized PC counterpart.

12. Conclusion

12.1 Summary

This project successfully demonstrated the feasibility of deploying a specialized **Convolutional Neural Network (CNN)** on the **STM32F746G-DISCO** platform. By leveraging the **MobileNetV2** backbone within a **FOMO (Faster Objects, More Objects)** architecture, we established a real-time monitoring solution for fruit fly detection.

Key Project Outcomes:

- **Hardware Efficiency:** The model was successfully optimized for the STM32's **1MB Flash** and **320KB RAM** constraints. By utilizing **INT8 quantization**, we achieved a footprint small enough to run without external memory, maintaining a high level of portability.
- **Predictable Performance:** The system exhibited remarkable determinism. With an average inference time of **461.92 ms** and a standard deviation of only **±4.95 ms**, the detector is suitable for continuous, stable environmental logging.
- **Detection Trade-offs:**
    - **Recall (72.83%):** The model proved highly capable of identifying fruit flies even in low-resolution and quantized environments.
    - **Precision (47.52%):** As anticipated, the aggressive quantization and small model size led to a higher rate of false positives, as background textures were occasionally misidentified as fly features.
- **Parity and Portability:** Comparison between PC-based simulation and the **TFLite Micro** implementation on the STM32 showed a near-exact match in detection logic, validating that the quantization process preserved the core functional intelligence of the model.

12.2 Lessons Learned

This project highlighted the distinct gap between theoretical machine learning on a PC and practical deployment on constrained embedded systems. The following key technical insights were established:

- **The "Black Box" Nature of Embedded Inference:** Unlike PC-based training where memory is virtually infinite and errors are verbose, the STM32 fails silently when limits are breached. We learned that static memory allocation is non-negotiable. The "Arena Size is too small" error was not merely a configuration issue but a lesson in memory mapping; the tensor arena

(200KB) had to be carefully balanced against the stack size required for UART communication buffers.

- **The Cost of Quantization**: While Post-Training Quantization (PTQ) reduced the model size by ~90%, it was the primary driver of the precision drop (High False Positives). We observed that the Representative Dataset used during conversion is critical. If the calibration images do not perfectly represent the noise floor of the target environment, the INT8 model becomes hypersensitive to background textures.

12.3 Future Improvements

- **Dataset Expansion:** The current dataset is heavily biased towards images containing flies. To improve the model's ability to reject background noise, the training set can be augmented with "Negative Samples"—images of the trap background containing debris, dust, or other insects that are not fruit flies.